

Islamic University of Technology

Lab 4: Amortized Analysis & Greedy Algorithm

CSE 4810: Algorithm Engineering
1A

ASH, ABT(PT)
June 18, 2026

General Instruction

This lab consists of two independent parts. Each part contains one **Amortized Analysis** task and one **Greedy Algorithm** task. For every task you must:

- Step 1.** Complete the `// TODO` sections in the provided C++ source file.
- Step 2.** Compile and run the program to produce a data `.txt` file.
- Step 3.** Run the matching Python plotter to generate a chart `.png`.

Files Provided

<code>amortized.cpp</code>	Two-Stack Queue skeleton
<code>greedy.cpp</code>	Huffman greedy-choice skeleton
<code>plotter.py</code>	Plots output of both tasks (auto-detects by filename)

Part A: Amortized Analysis of a Two-Stack Queue

A **queue** can be simulated using only two **stacks**: `stack_in` and `stack_out`. This is a classic example where the *worst-case* cost of a single operation is misleading, but the *amortized* cost tells the real story.

- **enqueue(val):** always cheap. Simply push `val` onto `stack_in`. Actual cost = 1.
- **dequeue():** if `stack_out` is empty, *every* element currently in `stack_in` must be popped and pushed onto `stack_out` before the front element can be returned. This single dequeue can therefore cost $O(n)$ in the worst case.

Despite this, using the **aggregate method** (or equivalently the **banker's method** with each element pre-paying for its own future transfer), the amortized cost of both `enqueue` and `dequeue` is $O(1)$. Intuitively: each element is pushed onto `stack_in` once, popped from `stack_in` once, pushed onto `stack_out` once, and popped from `stack_out` once — at most 4 units of work per element across its entire lifetime, no matter how the enqueues and dequeues are interleaved.

This produces a very distinctive cost profile: long stretches of cheap, constant-cost operations punctuated by occasional expensive *transfer spikes*, exactly like the binary counter's carry-propagation spikes from Lab 4 — except here the spike size depends on burst pattern rather than a fixed power of two.

0.1 TODO 1: Implement enqueue (val)

1. Push `val` onto `stack_in`.
2. Return the base `actual_cost` of 1.
(*Enqueue never touches `stack_out`, so it is always $O(1)$ in the worst case too.*)

0.2 TODO 2: Implement dequeue ()

1. If both stacks are empty, return 0 (underflow — nothing to dequeue).
2. If `stack_out` is empty, transfer everything: while `stack_in` is not empty, pop from `stack_in` and push onto `stack_out`, charging **+2** to `actual_cost` for each element moved (1 for the pop, 1 for the push).
(*If `stack_out` is already non-empty, skip this transfer entirely — that is what keeps the amortized cost low.*)
3. Pop the top element of `stack_out` — this is the true front of the queue.
4. Charge **+1** to `actual_cost` for this final pop.
5. Return `actual_cost`.

Sanity Check — Expected Output

```
1 --- TWO-STACK QUEUE SANITY CHECK ---
2 1st Dequeue Cost (Expected 11): 11
3 2nd Dequeue Cost (Expected 1) : 1
4 -----
```

Do not proceed to the plot until both lines match. The first dequeue after 5 enqueues must transfer all 5 elements (5 pops + 5 pushes = 10) plus the final pop (+1) = 11. The second dequeue pulls straight from `stack_out` with no transfer, costing only 1.

Compile & Run

```
1 g++ -O2 -o amortized amortized.cpp
2 ./amortized
3 python3 plotter.py data_queue.txt
```

0.3 What to Observe in the Plot

- **Blue scatter points** mark the *actual cost* of every single operation. You will see a dense baseline of cheap points (cost 1) interrupted by sharp upward spikes at the start of every dequeue burst, where one operation absorbs the cost of transferring an entire batch.
- **Red running-average line** stays low and nearly flat across the whole run, even though the blue spikes reach into the hundreds. This is the amortized cost made visible: the expensive spikes are "smoothed out" over many cheap operations.

- **Compare with Lab 4's Binary Counter plot.** Both show the same phenomenon (expensive operations bounded by cheap ones), but here the spike pattern is driven by burst size (`batch_size = 500`) rather than the fractal 2^i structure of bit flips.

Part B: Greedy Choice in Huffman Coding

Huffman coding builds an optimal prefix-free binary code by repeatedly merging the two *lowest-frequency* nodes in a forest of trees into a single parent node, until only one tree (the root) remains. The greedy choice — always combine the two cheapest available nodes — is provably optimal: any other merge order produces a tree with equal or greater weighted path length.

The interesting engineering question is not *whether* the algorithm is correct, but *how fast* you can repeatedly find the "two smallest" elements. You will implement and compare two approaches that make the identical greedy choice but with very different underlying data structures:

Approach	Data Structure	Why It Matters
buildHuffmanNaive	Plain <code>vector<Node*></code> , linear scan	Same greedy choice, slow extraction. Finding the two smallest nodes takes $O(N)$ each merge, giving $O(N^2)$ total.
buildHuffmanPQ	<code>priority_queue</code> (binary min-heap)	Same greedy choice, fast extraction. Each pop/push is $O(\log N)$, giving $O(N \log N)$ total.

0.4 TODO 1: Implement `buildHuffmanNaive(freqs)`

1. Loop until only 1 node remains in the `nodes` vector:
 - (a) Linearly scan `nodes` to find the index of the node with the absolute smallest `freq`.
 - (b) Linearly scan `nodes` again (skipping the first index found) to find the next-smallest `freq`.
 - (c) Create a new parent `Node` whose frequency is the sum of the two chosen frequencies.
 - (d) Add the parent's frequency to `total_internal_cost`.
 - (e) Erase the two merged nodes from `nodes` and insert the new parent node.
2. Return `total_internal_cost`.

[Hint] Erasing by index shifts the vector — erase the higher index first to avoid invalidating the lower one.

0.5 TODO 2: Implement `buildHuffmanPQ(freqs)`

1. Loop until `minHeap.size() == 1`:
 - (a) Pop the top of `minHeap` into `a` (the absolute smallest), then pop the new top into `b` (the next smallest).
 - (b) Create a new parent `Node` whose frequency is `a->freq + b->freq`.
 - (c) Add the parent's frequency to `total_internal_cost`.
 - (d) Push the new parent node back into `minHeap`.

2. Return `total_internal_cost`.

Both functions must return the same `total_internal_cost` for the same input frequencies. They are making the exact same greedy choice at every step — only the mechanism for finding "the two smallest" differs. If your two implementations disagree, one of your TODOs has a bug.

Compile & Run

```
1 g++ -O2 -o huffman greedy.cpp
2 ./huffman
3 python3 plotter.py data_huffman.txt
```

0.6 What to Observe in the Plot

- **Orange line (Naive $O(N^2)$):** Runtime grows quadratically as N increases from 1,000 to 30,000 — the curve should visibly bend upward, not stay straight.
- **Teal line (Priority Queue $O(N \log N)$):** Runtime grows far more slowly across the same range of N , and the gap between the two lines widens dramatically at the largest input sizes.
- **Think about it:** Even though both approaches produce an *identical* Huffman tree cost, only one of them will finish the $N = 30,000$ case in a reasonable amount of time. This is the core lesson of greedy algorithm engineering: the *correctness* of a greedy choice and the *efficiency* of implementing that choice are separate concerns.